

D1 Control Tower: the code, each file followed by its flowchart

The eight exhibits E1 to E8 exactly as they are in **d1/** of the kaveri-dock-lab repository. The code pages are generated from the files themselves, so what you read here is what runs. Plain, step-numbered Python with comments; **claude-sonnet-5-5** as the main model and **claude-haiku-4-5** for the small jobs. After each file comes a colour flowchart of the same code, with the points that matter for the CCAR-P exam marked by red dashed callouts.

How the files fit together

common.py is the shared desk: the two switches, the model names, the three inputs read from data/, the two tool schemas, the two tool functions, and the helpers every exhibit calls (text_of, trace, make_client, ask, ask_json, ask_async). It is chapter 0 of this document. **standin.py** is the scripted stand-in that answers when DRY_RUN=1; it is printed in the appendix and you do not need it to learn the patterns. E3 reuses E2's classify and gate. E8 reuses E5's make_plan, run_worker and synthesise.

Running the files

```
# live, from the repo root, with ANTHROPIC_API_KEY in a .env file there (git-ignored):
pip install anthropic
cd d1
python e1_augmented_llm.py
```

```
# on battery (no key, no cost), Windows Command Prompt:
set DRY_RUN=1
python e1_augmented_llm.py
set DRY_RUN=
```

```
# on battery, Linux or Mac:
DRY_RUN=1 python e1_augmented_llm.py
```

```
# everything at once on battery:  run_all.bat (Windows)  or  ./run_all.sh
# E7 branches on battery:      set D1_SCENARIO=truncate  (or refuse, exhaust) before running e7
```

How to read the flowcharts

Colour	Means
Yellow box	Code setting things up: constants, schemas, the tool functions. The model never sees these.
Green box	A model call. Each one is paid for live. Count the green boxes to count the bill.
Blue box	A code step: building the conversation, running a tool, counting, a plain-Python gate.
Red diamond	A decision on one field, usually stop_reason or a word against a set.
Purple box, light	A result.
Purple box, dark border	A hand-off to a human, or a stop.
Red dashed callout with a star	An exam point: the fact, law, or distractor a stem is likely to test.

Status

All eight files, and E7 under its three scenarios, run on battery with the traces shown in d1/README.md. E1 was run live against the API from your laptop with the earlier version of this file and produced the same two-turn trace. The rewritten files have not yet been run live; the control flow is unchanged, the wording of a live reply will vary.

Contents

0. common.py, the shared desk (code only)
1. E1 The augmented LLM (e1_augmented_llm.py) code, then flowchart
2. E2 Prompt chaining (e2_chaining.py) code, then flowchart

3. E3 Routing (`e3_routing.py`) code, then flowchart
 4. E4 Parallelization (`e4_parallel.py`) code, then flowchart
 5. E5 Orchestrator-workers (`e5_orchestrator_workers.py`) code, then flowchart
 6. E6 Evaluator-optimizer (`e6_evaluator_optimizer.py`) code, then flowchart
 7. E7 The agent loop (`e7_agent_loop.py`) code, then flowchart
 8. E8 Re-planning (`e8_replanning.py`) code, then flowchart
- Appendix. `standin.py`, the dry-run stand-in (code only)

Chapter 0 common.py: the shared desk

Read STEP A to STEP F once. Every exhibit that follows imports from here, so when a chapter says 'ask comes from common.py' or 'the tool is fake_status', this is where it lives.

```
# common.py - the shared desk for the eight Domain 1 exhibits.
#
# Every exhibit imports from here, so the things that never change live in one place:
# STEP A the two switches (DRY_RUN, D1_SCENARIO) and the optional .env loader
# STEP B which models to use, and how long a reply may be
# STEP C the three inputs: Padma's email, the refund claim, Meera's brief (read from data/)
# STEP D the two tool DESCRIPTIONS (schemas) the model may ask for
# STEP E the tools THEMSELVES: plain Python that answers from a canned world
# STEP F helpers: text_of(), trace(), make_client(), ask(), ask_json(), ask_async()
#
# DRY_RUN=1 -> make_client() returns the scripted stand-in from standin.py:
#             same control flow, canned replies, no network, no bill.
# unset      -> the real anthropic SDK. Needs: pip install anthropic
#             and ANTHROPIC_API_KEY in the environment, or in a .env file in the repo root (git-ignored).
import asyncio
import json
import os
import re
from pathlib import Path

# -----
# STEP A. Switches, and the optional .env loader
# -----

# If a .env file exists (next to this file, or in the repo root), copy its KEY=value lines into the
# environment. A variable that is already set wins. The file is git-ignored: never commit the key.
for env_file in (Path(__file__).with_name(".env"), Path(__file__).resolve().parent.parent / ".env"):
    if env_file.exists():
        for line in env_file.read_text().splitlines():
            line = line.strip()
            if line == "" or line.startswith("#") or "=" not in line:
                continue
            key, value = line.split("=", 1)
            os.environ.setdefault(key.strip(), value.strip().strip('"').strip("'"))
DRY_RUN = os.environ.get("DRY_RUN") == "1" # battery mode: stand-in client, zero paid calls
SCENARIO = os.environ.get("D1_SCENARIO", "default") # steers the stand-in's final reply in E7

# -----
# STEP B. Models and reply length
# -----

MODEL_MAIN = "claude-sonnet-5-5" # judgement, planning, drafting, synthesis
MODEL_SMALL = "claude-haiku-4-5" # one-word sorts, small workers, short apologies
MAX_TOKENS = 1024 # the longest reply we accept; thinking tokens count toward it
# Production would put refunds and planning on the Opus tier (claude-opus-5-5).
# Sonnet is used here so a learning run stays cheap. Change the one constant above to switch.
# -----
# STEP C. The three inputs, read from the data/ folder
# -----

DATA = Path(__file__).with_name("data")
EMAIL = (DATA / "padma_email.txt").read_text(encoding="utf-8") # E1, E2, E3, E6
CLAIM_TEXT = (DATA / "refund_claim.txt").read_text(encoding="utf-8") # E4 voting
BRIEF = (DATA / "brief_kf2481.txt").read_text(encoding="utf-8") # E5, E8

# -----
# STEP D. The two tool descriptions. A schema is a phone-book entry:
#             name, what it does, what input it needs. It is NOT the code.
# -----
TOOLS_KAVERI = [
    {
        "name": "get_shipment_status",
        "description": "Live status for one Kaveri consignment id (e.g. KF-2481).",
        "input_schema": {
            "type": "object",
            "properties": {"consignment_id": {"type": "string"}},
            "required": ["consignment_id"],
        },
    },
]
```

```

    {
        "name": "find_alternate_carriers",
        "description": "Carriers that can take a lane (e.g. Pune-Chennai) within 48 hours.",
        "input_schema": {
            "type": "object",
            "properties": {"lane": {"type": "string"}},
            "required": ["lane"],
        },
    },
]
# -----
# STEP E. The tools themselves. Plain Python. The model never sees this code.
#       In production these would query the depot database or the MCP server.
# -----

def fake_status(consignment_id):
    return {"id": consignment_id, "status": "CUSTOMS_HOLD", "missing": "e-way bill", "days_late": 3}

def fake_carriers(lane):
    return ["BlueDart", "Delhivery"]
# -----
# STEP F. Helpers every exhibit uses
# -----

def text_of(reply):
    """Collect the text blocks only. Never read content[0]: a thinking block may come first."""
    answer = ""
    for block in reply.content:
        if block.type == "text":
            answer = answer + block.text
    return answer

def trace(turn, reply):
    """Print one line per call: the turn number, the stop_reason, and any tools the model asked for."""
    tool_names = []
    for block in reply.content:
        if block.type == "tool_use":
            tool_names.append(block.name)
    line = "# turn " + str(turn) + " stop_reason=" + str(reply.stop_reason)
    if tool_names:
        line = line + " (" + ", ".join(tool_names) + ")"
    print(line)

def _sdk_kwargs():
    """An org-level key must name a workspace on every request. Set ANTHROPIC_WORKSPACE_ID in .env,
    or use a key created inside a workspace, in which case this header is not needed."""
    workspace = os.environ.get("ANTHROPIC_WORKSPACE_ID")
    if workspace:
        return {"default_headers": {"anthropic-workspace-id": workspace}}
    return {}

def make_client():
    """The stand-in on battery, the real SDK live. The exhibits do not know or care which one they got."""
    if DRY_RUN:
        from standin import FakeClient
        return FakeClient()
    import anthropic # only imported on the paid path
    return anthropic.Anthropic(**_sdk_kwargs())

def ask(model, question, system=None, max_tokens=MAX_TOKENS):
    """One call, text in, text out. No tools. Checks stop_reason before trusting the text."""
    arguments = {
        "model": model,
        "max_tokens": max_tokens,
        "messages": [{"role": "user", "content": question}],
    }
    if system is not None:
        arguments["system"] = system

    reply = make_client().messages.create(**arguments)
    if reply.stop_reason == "refusal":
        raise RuntimeError("The model declined to answer.")
    return text_of(reply).strip()

def ask_json(model, question, max_tokens=MAX_TOKENS):
    """ask(), then parse the reply as JSON. Models sometimes wrap JSON in a ``json fence; strip it."""
    raw = ask(model, question, max_tokens=max_tokens)
    if raw.startswith("```"):
        lines = raw.split("\n")

```

```

        lines = lines[1:]                # drop the opening ```json line
        if lines and lines[-1].strip() == "```":
            lines = lines[:-1]           # drop the closing ``` line
        raw = "\n".join(lines)
        return json.loads(raw)
async def ask_async(model, question, max_tokens=256):
    """The async twin of ask(), for E4. On battery it sleeps 0.2 s so the slowest-scout lesson is visible."""
    if DRY_RUN:
        from standin import canned_text
        await asyncio.sleep(0.2)
        return canned_text(None, question)

    import anthropic
    client = anthropic.AsyncAnthropic(**_sdk_kwargs())
    reply = await client.messages.create(
        model=model,
        max_tokens=max_tokens,
        messages=[{"role": "user", "content": question}],
    )
    if reply.stop_reason == "refusal":
        raise RuntimeError("The model declined to answer.")
    return text_of(reply).strip()

```

E1 The augmented LLM: the code

One cell. One call, one tool, at most one round-trip.

```
# E1 · The augmented LLM: KAVI triages one exception (objective 1.3a, the cell every organism is built from)
#
# One call, one tool, at most one round-trip. The model may ask us to look the shipment up.
# If it does, we run the lookup and send the fact back. Then it answers.
#
# Run on battery: DRY_RUN=1 python e1_augmented_llm.py          (Windows: set DRY_RUN=1 first)
# Run live:      python e1_augmented_llm.py                    (needs ANTHROPIC_API_KEY, see common.py)

import json
from common import make_client, trace, text_of, fake_status, MODEL_MAIN, MAX_TOKENS, EMAIL, TOOLS_KAVERI,
    DRY_RUN
# -----
# STEP 0. Things we decide before talking to the model
# (MODEL_MAIN, MAX_TOKENS and EMAIL come from common.py)
# -----

# The ONE tool the model is allowed to ask for. This is only a description: name, what it does,
# what input it needs. The model cannot run it. We run it.
TOOLS = [TOOLS_KAVERI[0]]          # get_shipment_status

# The instructions taped to the model's desk.
SYSTEM = (
    "You triage Kaveri shipment exceptions. "
    "Reply as JSON {category, urgency} and nothing else. "
    "Categories: delay, damage, customs, refund, address, other."
)
# STEP 1. The tool itself lives in common.py as fake_status(): plain Python. The model never sees it.

def triage(email_text):

    # STEP 2. Connect. DRY_RUN=1 gives the scripted stand-in; otherwise the real SDK.
    client = make_client()

    # STEP 3. Start the conversation with one message: the email. This is page 1 of the case file.
    conversation = []
    conversation.append({"role": "user", "content": email_text})

    # STEP 4. First call to the model.
    reply = client.messages.create(
        model=MODEL_MAIN,
        max_tokens=MAX_TOKENS,
        system=SYSTEM,
        tools=TOOLS,
        messages=conversation,
    )
    trace(1, reply)          # prints: # turn 1 stop_reason=... (tool names)
    # STEP 5. Did the model ask for the tool? "tool_use" means: "I need a lookup before I can answer."
    if reply.stop_reason == "tool_use":

        # 5a. Find the tool request inside the reply. A reply is a list of blocks.
        tool_request = None
        for block in reply.content:
            if block.type == "tool_use":
                tool_request = block
                break

        # 5b. Run the tool ourselves, with the input the model filled in.
        consignment_id = tool_request.input["consignment_id"]
        lookup_result = fake_status(consignment_id)
        # 5c. Add the model's own request to the conversation: page 2.
        # The API remembers nothing between calls, so we must send its own words back.
        conversation.append({"role": "assistant", "content": reply.content})

        # 5d. Add the tool's answer: page 3. It is tagged with the request's id so the model knows
        # which question this answers. The API wants text, so the dict becomes a JSON string.
        tool_answer = {
            "type": "tool_result",
            "tool_use_id": tool_request.id,
            "content": json.dumps(lookup_result),
        }
        conversation.append({"role": "user", "content": [tool_answer]})
        # 5e. Second call. Same five arguments. The conversation is now 3 pages long.
        reply = client.messages.create(
```

```

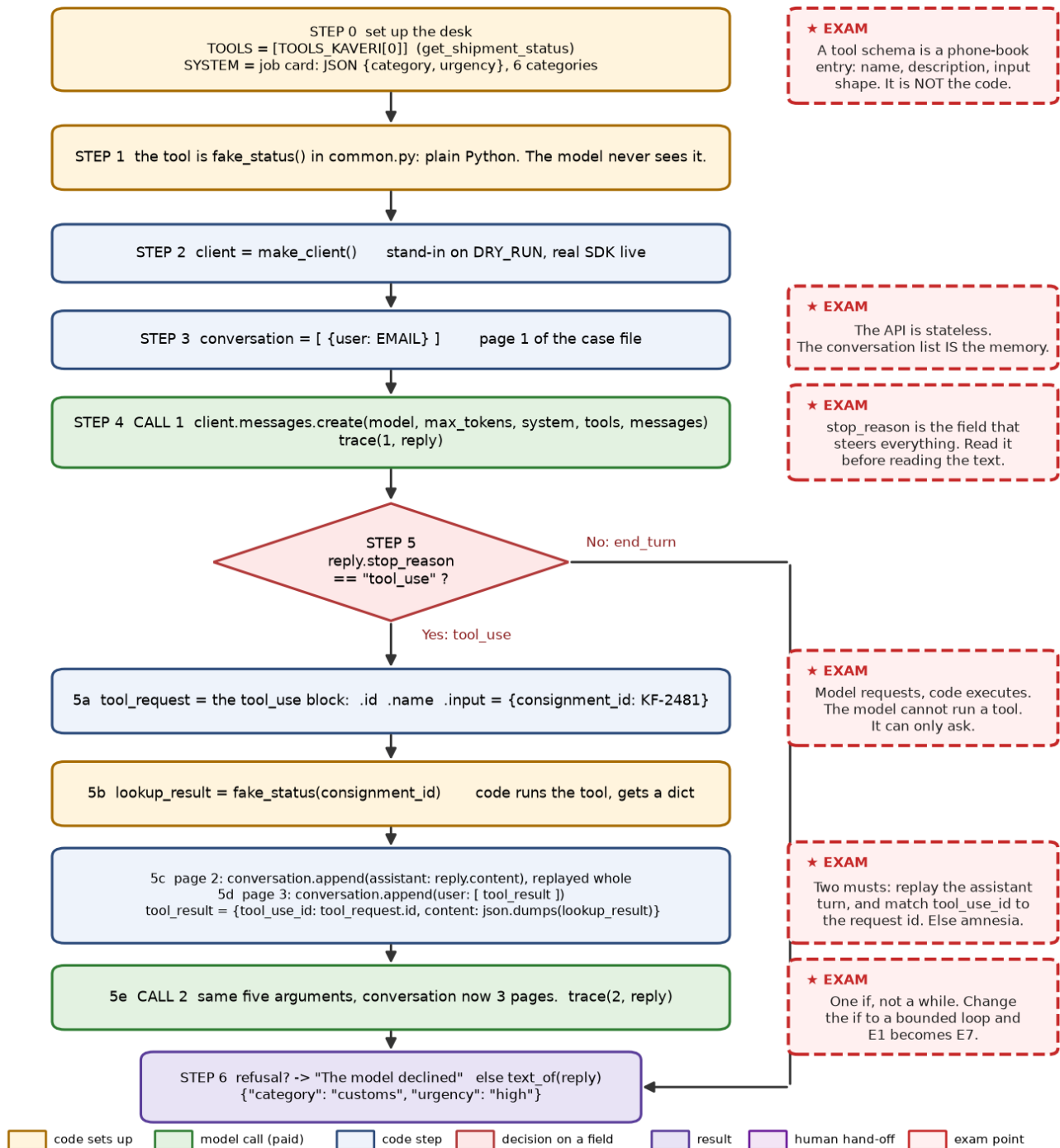
        model=MODEL_MAIN,
        max_tokens=MAX_TOKENS,
        system=SYSTEM,
        tools=TOOLS,
        messages=conversation,
    )
    trace(2, reply)

# STEP 6. Read the final answer. A refusal is not an answer; otherwise collect the text blocks only.
if reply.stop_reason == "refusal":
    return "The model declined to answer."
return text_of(reply)
if __name__ == "__main__":
    if DRY_RUN:
        print("# E1 · augmented LLM · DRY_RUN: scripted stand-in, 0 paid calls")
    else:
        print("# E1 · augmented LLM · LIVE · " + MODEL_MAIN)
    print(triage(EMAIL))

```

E1 The augmented LLM: the flowchart

E1 e1_augmented_llm.py: one cell, at most one round-trip



Exam points on this chart

- ★ A tool schema is a description, not code. The model requests; your code executes.
- ★ The API is stateless: the conversation list is the memory. Replay the assistant turn and match tool_use_id.
- ★ stop_reason steers everything. E1 uses one if; E7 turns that if into a bounded loop.

E2 Prompt chaining: the code

Two desks in a fixed order with a plain-Python gate between them.

```
# E2 · Prompt chaining: two desks in a fixed order, a checklist between them (objective 1.3b, workflow 1)
#
# Desk 1: read the email, say ONE word: what kind of problem is this?
# Gate: plain Python checks that word is on the approved list. If not, stop the line.
# Desk 2: knowing the category, write a short courteous reply.
# The CODE decides the order. The model never gets to change it. That is Law 2.
#
# Run on battery: DRY_RUN=1 python e2_chaining.py          Run live: python e2_chaining.py

from common import ask, MODEL_SMALL, MODEL_MAIN, EMAIL, DRY_RUN

# -----
# STEP 0. The laminated checklist on the wall between the two desks.
# Only these six words are allowed to pass from desk 1 to desk 2.
# -----
VALID_CATEGORIES = {"delay", "damage", "customs", "refund", "address", "other"}

# -----
# STEP 1. DESK 1: classify. One word only, on the small model.
# -----

def classify(email_text):
    question = (
        "Read this customer email and reply with ONE word only, the category of the problem. "
        "Choose from: delay, damage, customs, refund, address, other. "
        "No explanation, no punctuation, just the word.\n\n"
        + email_text
    )
    word = ask(MODEL_SMALL, question)
    return word.lower()          # "Customs" and "customs" become the same thing
# -----
# STEP 2. THE GATE: plain Python. No model, no cost.
# If desk 1 mumbled something that is not on the checklist, stop the line here.
# -----

def gate(category):
    if category not in VALID_CATEGORIES:
        raise ValueError("bad category: " + category)
    return category

# -----
# STEP 3. DESK 2: draft the reply. It is TOLD the category; it does not re-decide it.
# -----

def draft_reply(email_text, category):
    question = (
        "This is a " + category + " exception at Kaveri Freight. "
        "Draft a courteous reply to the customer, under 120 words. "
        "Do not promise any refund or compensation.\n\n"
        + email_text
    )
    return ask(MODEL_MAIN, question)

if __name__ == "__main__":
    print("# E2 · prompt chaining · " + ("DRY_RUN" if DRY_RUN else "LIVE"))
    # STEP 4. Run the line in the order the code fixes: classify -> gate -> draft.
    print("=== Desk 1: classify ===")
    category = classify(EMAIL)
    print("Desk 1 said:", repr(category))

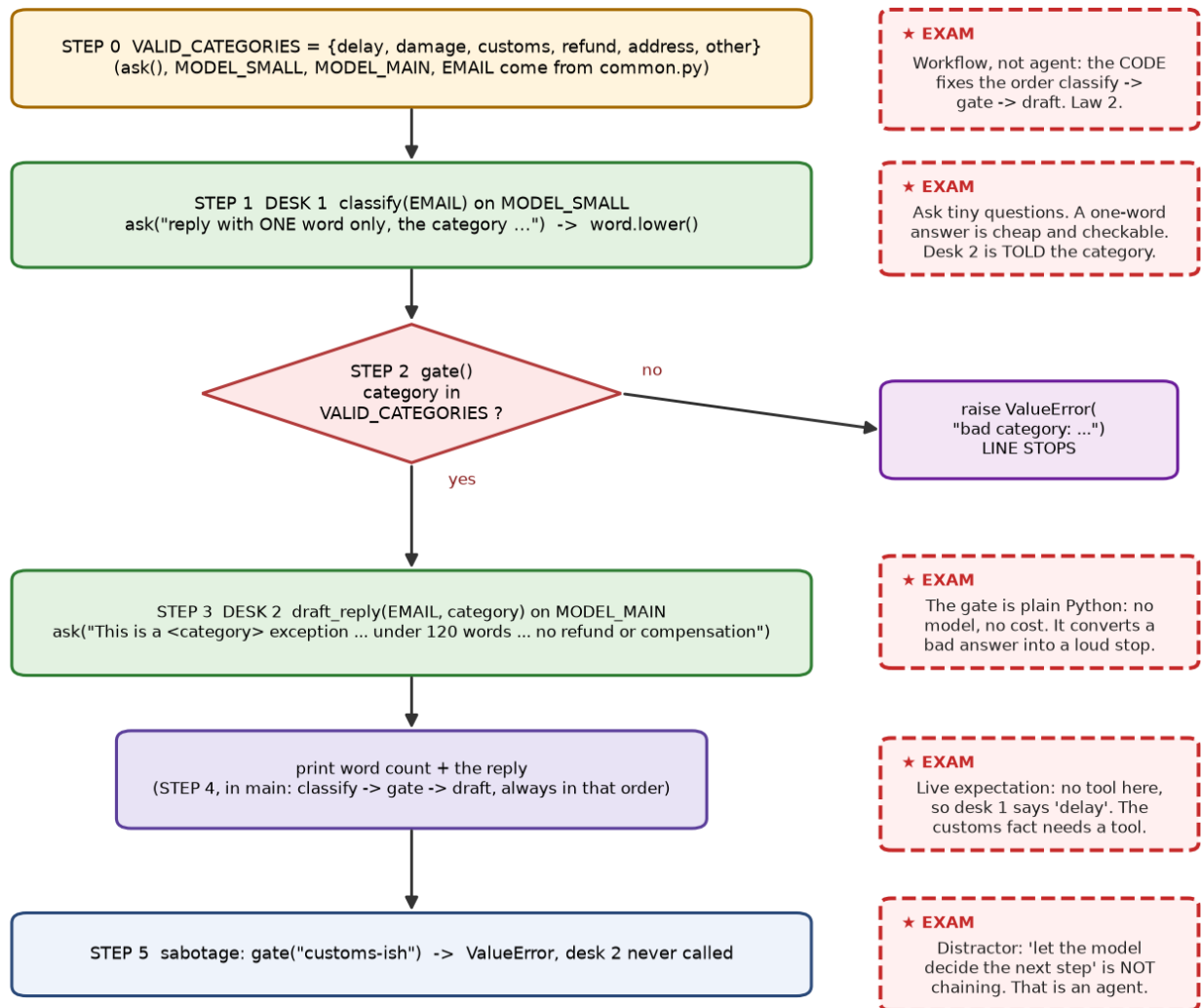
    print()
    print("=== Gate: check the word against the list ===")
    category = gate(category)
    print("Gate passed:", category, "is on the approved list. Line continues.")

    print()
    print("=== Desk 2: draft ===")
    reply = draft_reply(EMAIL, category)
    print("Desk 2 wrote", len(reply.split()), "words:")
    print()
    print(reply)
```

```
# STEP 5. Sabotage test: hand the gate a word that is not on the list. No model call here.
print()
print("=== Sabotage: feed the gate a word that is not on the list ===")
try:
    gate("customs-ish")
except ValueError as error:
    print("Gate stopped the line:", error)
    print("Desk 2 was never called. No reply was written for a nonsense category.")
```

E2 Prompt chaining: the flowchart

E2 e2_chaining.py: fixed order, a code gate between the desks



code sets up
model call (paid)
code step
decision on a field
result
human hand-off
exam point

Exam points on this chart

- ★ Workflow, not agent: the code fixes the order. Law 2.
- ★ Ask tiny questions; a one-word answer is cheap and checkable. Desk 2 is told the category.
- ★ The gate costs nothing and converts a bad answer into a loud stop.

E3 Routing: the code

A cheap nurse call picks the ward; the ward picks the model.

```
# E3 · Routing: a cheap nurse at the door, specialist wards behind it (objective 1.3b, workflow 2)
#
# Nurse: one cheap call says ONE word: which kind of problem is this? (E2's classify + gate, reused)
# Table: the word picks WHICH specialist, and which MODEL, handles the email.
# Ward: ONE call to that specialist. It never saw a menu. The code chose.
# Routing is where model selection stops being one decision and becomes a decision PER category.
#
# Run on battery: DRY_RUN=1 python e3_routing.py          Run live: python e3_routing.py

from common import ask, MODEL_MAIN, MODEL_SMALL, EMAIL, DRY_RUN
from e2_chaining import classify, gate          # reuse E2's nurse and checklist
# -----
# STEP 0. The wards. Each category maps to (which model, the ward's instructions).
# This table IS the router. Add a ward by adding a row. No model is involved in the choice.
# -----

SPECIALISTS = {
    "refund": (MODEL_MAIN, "You resolve refund disputes at Kaveri Freight. Acknowledge, say what you will
        verify, never promise money."),
    "customs": (MODEL_MAIN, "You explain customs holds and next steps for Indian import consignments at Kaveri
        Freight."),
    "delay": (MODEL_SMALL, "You write short delay apologies for Kaveri Freight with a new ETA. Under 80
        words."),
}
# Production would put the refund ward on the Opus tier. MODEL_MAIN is Sonnet here to keep learning runs cheap.
# If the nurse says a word that has no ward (for example "address"), send it here instead of crashing.
DEFAULT_WARD = "delay"

# -----
# STEP 1. Table lookup: plain Python. An unmapped word goes to a safe ward, not a KeyError at 2 a.m.
# -----

def pick_ward(category):
    if category in SPECIALISTS:
        return category
    return DEFAULT_WARD
# -----
# STEP 2. The router: email in, specialist's reply out.
# -----

def route(email_text):
    # 2a. The nurse looks once and says one word. The gate checks it.
    category = gate(classify(email_text))

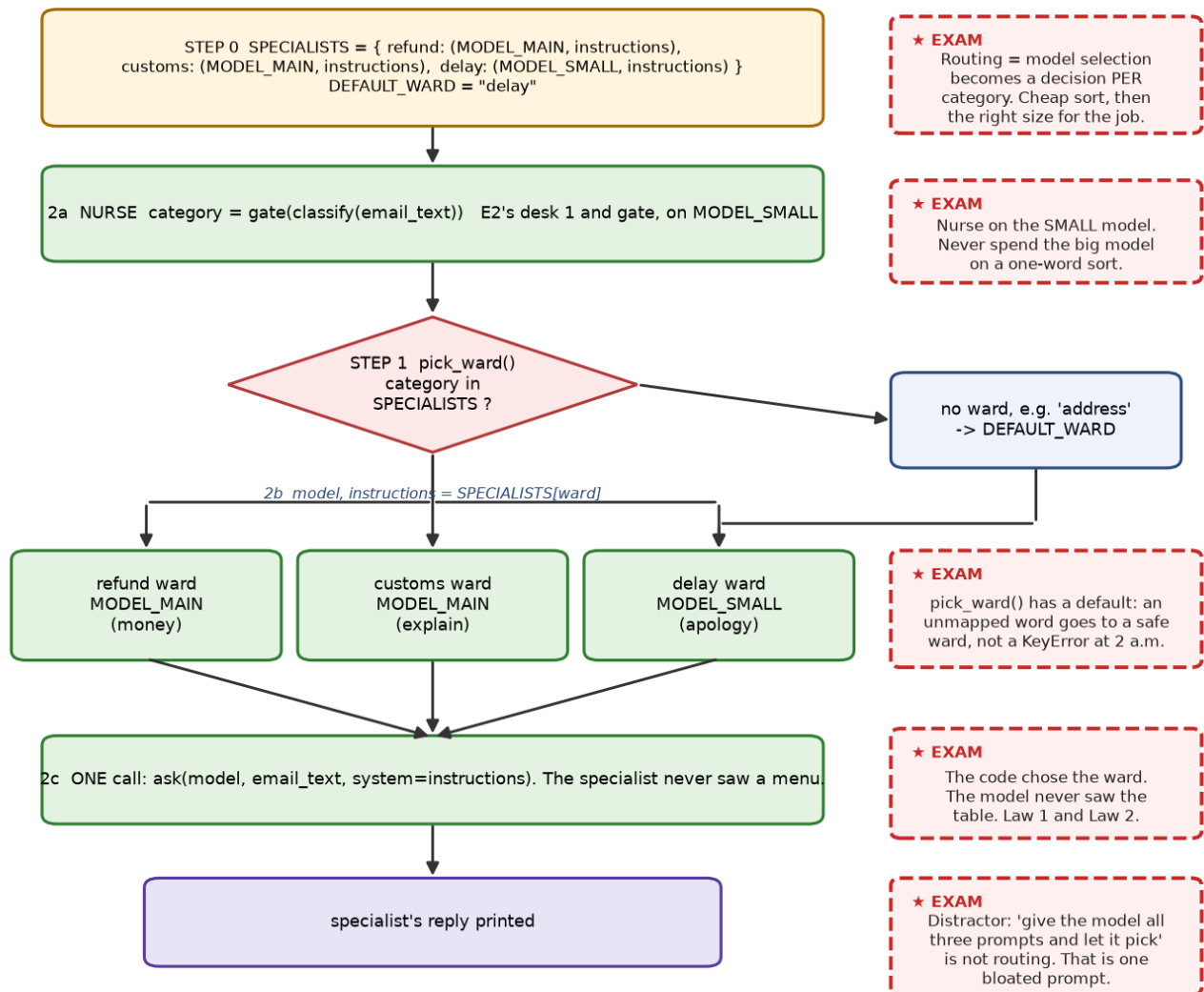
    # 2b. The word picks the ward; the ward brings its own model and instructions.
    ward = pick_ward(category)
    model, instructions = SPECIALISTS[ward]
    print("Nurse said:", repr(category), "-> ward:", ward, "-> model:", model)
    # 2c. ONE call to the chosen specialist. Its instructions go in the system prompt.
    return ask(model, email_text, system=instructions)

if __name__ == "__main__":
    print("# E3 · routing · " + ("DRY_RUN" if DRY_RUN else "LIVE"))

    # STEP 3. Route Padma's email.
    print("=== Routing Padma's email ===")
    specialist_reply = route(EMAIL)
    print()
    print("Specialist wrote:")
    print(specialist_reply)
    # STEP 4. Show the default ward. No model call: just the table lookup.
    print()
    print("=== A word with no ward ===")
    word = "address"
    ward = pick_ward(word)
    print(repr(word), "has no ward, so it goes to", repr(ward), "on model", SPECIALISTS[ward][0])
```

E3 Routing: the flowchart

E3 e3_routing.py: a cheap nurse picks the ward, the ward picks the model



code sets up model call (paid) code step decision on a field result human hand-off exam point

Exam points on this chart

- ★ Routing is where model selection becomes a decision per category.
- ★ Nurse on the small model; never spend the big model on a one-word sort.
- ★ pick_ward() has a default: an unmapped word is a safe fallback, not a KeyError.

E4 Parallelization: the code

Sectioning for coverage, voting for confidence. Same trick, opposite purposes.

```
# E4 · Parallelization: three scouts, or three judges (objective 1.3b, workflow 3)
#
# Sectioning: three DIFFERENT questions at once (three scouts on three roads). Purpose: coverage, speed.
# Voting: the SAME question three times (three judges). Purpose: confidence. Costs 3x.
# Same trick, simultaneity, opposite purposes. The CODE drew the fork; no model chose to parallelize.
#
# Run on battery: DRY_RUN=1 python e4_parallel.py (each call sleeps 0.2 s) Run live: python
e4_parallel.py

import asyncio
import time
from common import ask_async, MODEL_SMALL, MODEL_MAIN, CLAIM_TEXT, DRY_RUN
# -----
# STEP 0. Things we decide before talking to the model
# (ask_async in common.py uses the async SDK client, so several calls can be in flight at once)
# -----

CONSIGNMENT = "KF-2481"
CARRIERS = ["BlueDart", "Delhivery", "IndiaPost"]

# -----
# STEP 1. SECTIONING: three different questions, all at once
# -----
async def check_one_carrier(carrier):
    question = (
        "In one line, give a plausible ETA for consignment " + CONSIGNMENT
        + " on the Pune-Chennai lane if re-routed via " + carrier + ". "
        "Assume it is currently held at customs. Start your line with the carrier name."
    )
    return await ask_async(MODEL_SMALL, question)

async def sectioning():
    # asyncio.gather starts all three and waits for the slowest. Wall clock = the slowest scout.
    return await asyncio.gather(
        check_one_carrier(CARRIERS[0]),
        check_one_carrier(CARRIERS[1]),
        check_one_carrier(CARRIERS[2]),
    )

async def sequential():
    # The same three questions one after another, for comparison. Wall clock = the sum.
    results = []
    for carrier in CARRIERS:
        results.append(await check_one_carrier(carrier))
    return results

# -----
# STEP 2. VOTING: the same question three times, majority wins
# -----
async def one_vote():
    question = (
        "You are a claims reviewer at Kaveri Freight. Read this refund claim and reply with "
        "exactly one word, APPROVE or REJECT. No explanation.\n\n" + CLAIM_TEXT
    )
    return await ask_async(MODEL_MAIN, question)

def count_votes(votes):
    # Plain Python counts. No fourth model call to decide the majority.
    approve_count = 0
    reject_count = 0
    for vote in votes:
        if "APPROVE" in vote.upper():
            approve_count = approve_count + 1
        else:
            reject_count = reject_count + 1
    if approve_count > reject_count:
        return "APPROVE"
    return "REJECT"

async def voting():
    votes = await asyncio.gather(one_vote(), one_vote(), one_vote())
    return votes, count_votes(votes)
```

```

# -----
# STEP 3. Run both, and time the sectioning against the sequential version
# -----

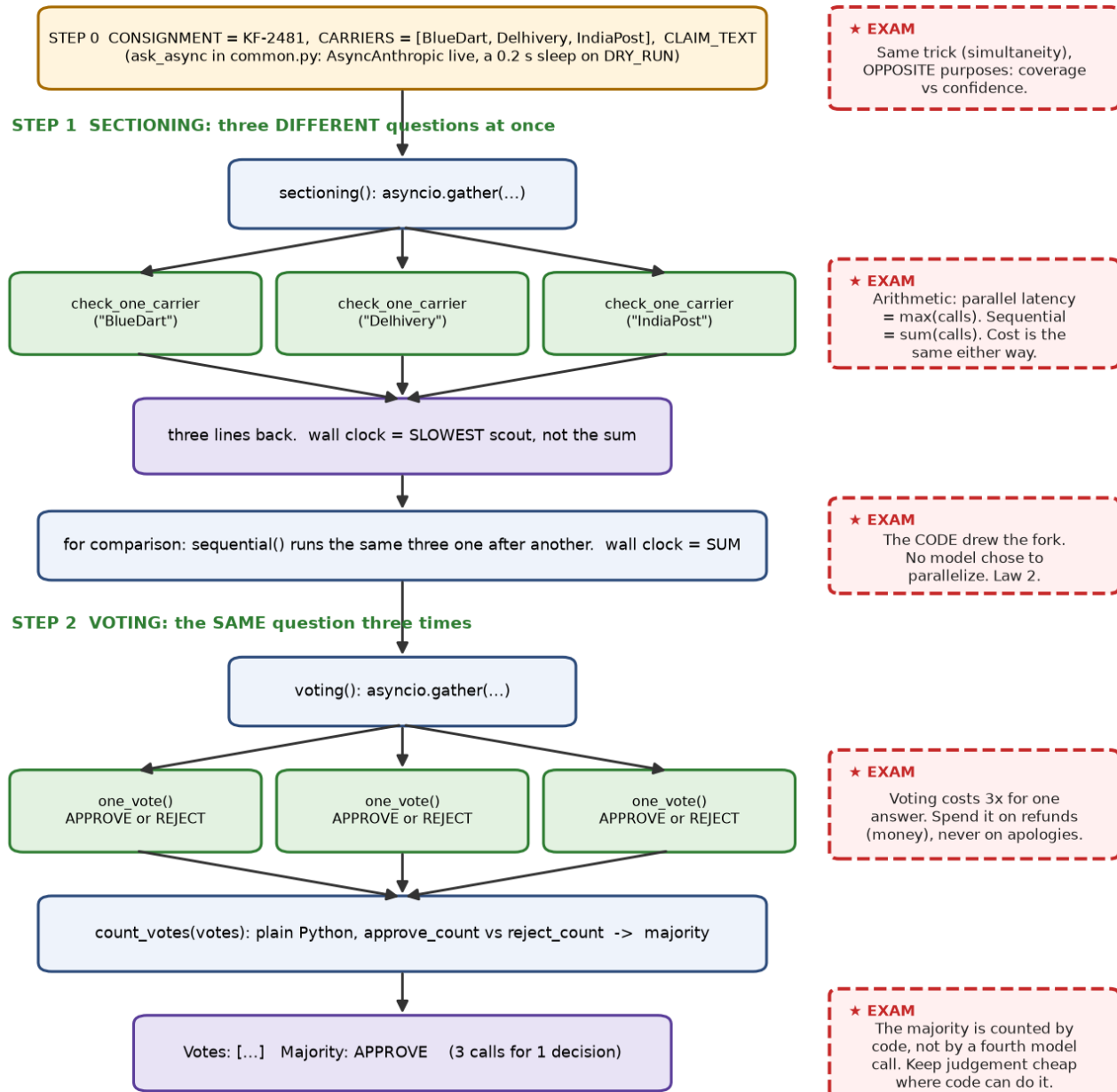
async def main():
    print("=== Sectioning: three scouts at once ===")
    start = time.perf_counter()
    scouts = await sectioning()
    parallel_seconds = time.perf_counter() - start
    for line in scouts:
        print("  scout ->", line)
    print()
    print("=== The same three scouts, one after another ===")
    start = time.perf_counter()
    await sequential()
    sequential_seconds = time.perf_counter() - start
    print("Parallel took", round(parallel_seconds, 2), "s (about the slowest single call).")
    print("Sequential took", round(sequential_seconds, 2), "s (about the sum of all three).")

    print()
    print("=== Voting: three judges on one refund claim ===")
    votes, majority = await voting()
    print("Votes:", list(votes))
    print("Majority:", majority)
    print("Note: three calls for one decision. Spend that on refunds, not on apologies.")
if __name__ == "__main__":
    print("# E4 · parallelization · " + ("DRY_RUN (each call sleeps 0.2 s)" if DRY_RUN else "LIVE"))
    asyncio.run(main())

```

E4 Parallelization: the flowchart

E4 e4_parallel.py: sectioning for coverage, voting for confidence



code sets up model call (paid) code step decision on a field result human hand-off exam point

Exam points on this chart

- ★ Parallel latency = max(calls); sequential = sum(calls). Cost is the same either way.
- ★ Voting costs 3x for one decision: spend it on money, not on apologies.
- ★ The code drew the fork and the code counts the votes.

E5 Orchestrator-workers: the code

The foreman model writes the plan; code steers the workers; synthesis writes one recommendation.

```
# E5 · Orchestrator-workers: the site foreman writes the plan himself (objective 1.5, and workflow 4 of 1.3b)
#
# Foreman (model): reads the brief, writes a list of subtasks as JSON.
# Workers (model): one call per subtask. Each sees ONLY the findings it declared it needs, plus the depot
# facts.
# Synthesis (model): sees all findings, writes ONE recommendation.
# The SHAPE (plan -> work -> synthesise) is fixed code. The CONTENT of the plan is the model's.
#
# Run on battery: DRY_RUN=1 python e5_orchestrator_workers.py      Run live: python e5_orchestrator_workers.py

import json
from common import ask, ask_json, fake_status, fake_carriers, MODEL_MAIN, MODEL_SMALL, BRIEF, DRY_RUN
# -----
# STEP 0. The facts the depot system can supply. The CODE hands these to the workers.
# Workers reason over facts; they do not invent them.
# -----

FACTS = {
    "shipment": fake_status("KF-2481"),
    "carriers_on_lane": fake_carriers("Pune-Chennai"),
}

# -----
# STEP 1. FOREMAN: the model writes the plan
# -----
def make_plan(brief):
    question = (
        "Brief:\n" + brief + "\n\n"
        "Break this into 2 to 5 subtasks. Reply with JSON only, a list like:\n"
        '["id": 1, "task": "status check", "needs": []], {"id": 2, "task": "carrier options", "needs": [1]}\n'
        '"needs" lists the ids of earlier subtasks whose findings this one depends on.'
    )
    return ask_json(MODEL_MAIN, question)

# -----
# STEP 2. WORKER: one subtask, on the small model.
# It is shown ONLY the findings it declared it needs (context isolation), plus the depot facts.
# -----
def run_worker(subtask, findings_so_far):
    known = {}
    for needed_id in subtask["needs"]:
        if needed_id in findings_so_far:
            known[needed_id] = findings_so_far[needed_id]

    question = (
        "You are a worker at Kaveri Freight. Do this one subtask and reply in one or two lines.\n"
        "Subtask: " + subtask["task"] + "\n"
        "Known findings from earlier subtasks: " + json.dumps(known) + "\n"
        "Depot facts you may use: " + json.dumps(FACTS)
    )
    return ask(MODEL_SMALL, question)

# -----
# STEP 3. SYNTHESIS: sees everything, writes ONE recommendation
# -----
def synthesise(brief, findings):
    question = (
        "Brief:\n" + brief + "\n\n"
        "Findings from the workers:\n" + json.dumps(findings, indent=2) + "\n\n"
        "Write ONE recommendation for Meera's desk, three sentences at most, starting with RECOMMENDATION:"
    )
    return ask(MODEL_MAIN, question)

# -----
# STEP 4. The fixed shape: plan, then work in plan order, then synthesise
# -----

def orchestrate(brief):
    plan = make_plan(brief)
    print("Foreman's plan:", json.dumps(plan))

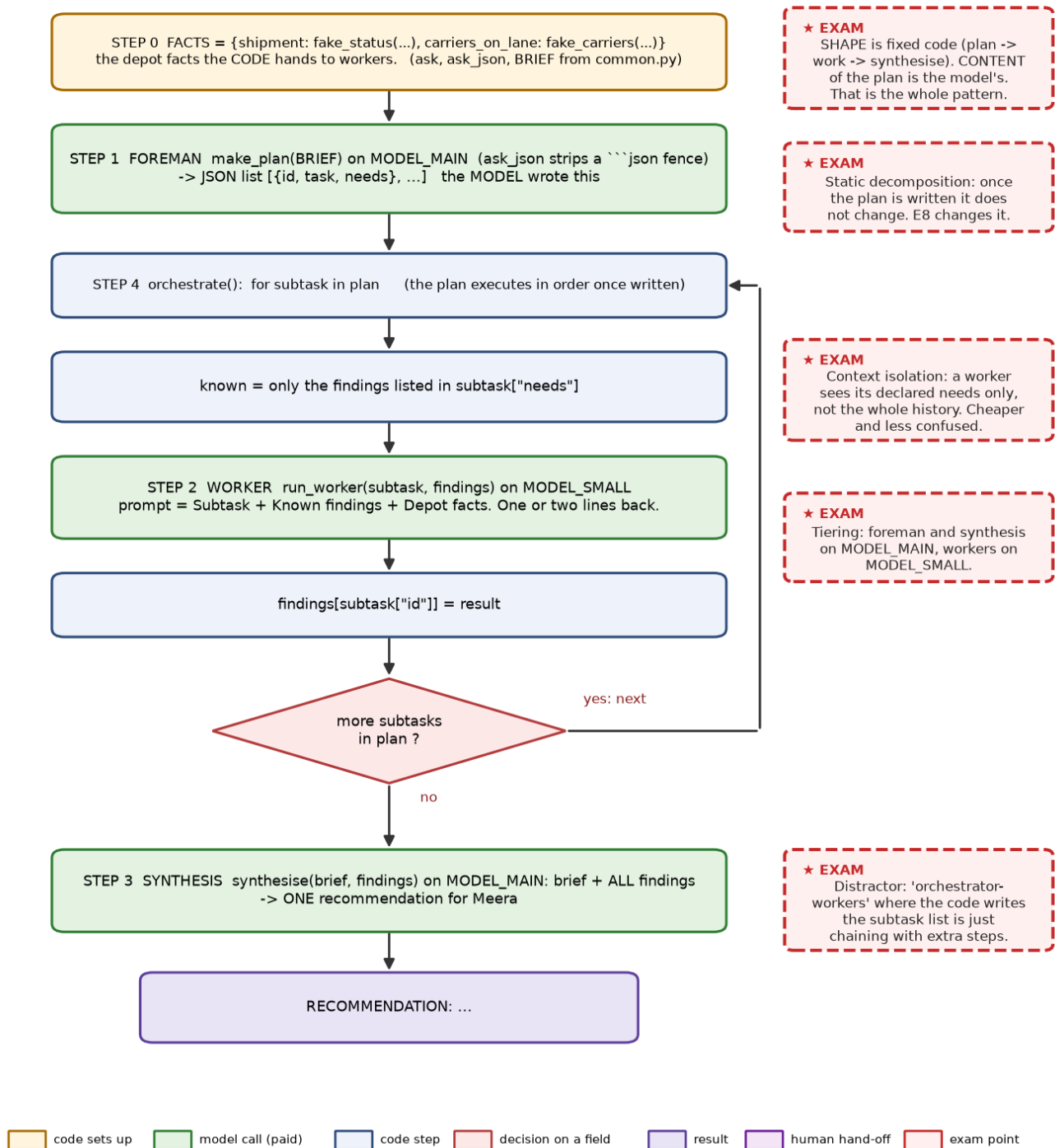
    findings = {}
    for subtask in plan:
        # the plan executes in order once written
```

```
        result = run_worker(subtask, findings)
        findings[subtask["id"]] = result
        print("Worker", subtask["id"], "(" + subtask["task"] + ") needs", subtask["needs"], "->", result)
    return synthesise(brief, findings)
```

```
if __name__ == "__main__":
    print("# E5 · orchestrator-workers · " + ("DRY_RUN" if DRY_RUN else "LIVE"))
    print(orchestrate(BRIEF))
```

E5 Orchestrator-workers: the flowchart

E5 e5_orchestrator_workers.py: the foreman writes the plan, code steers the work



Exam points on this chart

- ★ Shape is fixed code; content of the plan is the model's. Static decomposition.
- ★ Context isolation: a worker sees only its declared needs, plus facts the code supplies.
- ★ Tiering: foreman and synthesis on MODEL_MAIN, workers on MODEL_SMALL.

E6 Evaluator-optimizer: the code

Writer and editor, with a page limit.

```
# E6 · Evaluator-optimizer: a writer and an editor, with a page limit (objective 1.3b, workflow 5)
#
# Writer (model): drafts a reply.
# Editor (model): checks it against a rubric. Says PASS, or one line of criticism.
# Writer again: revises using the criticism. Repeat.
# Two organs: MAX_ROUNDS is the budget, PASS is the exit. E7's agent loop has exactly these two, grown larger.
#
# Run on battery: DRY_RUN=1 python e6_evaluator_optimizer.py      Run live: python e6_evaluator_optimizer.py

from common import ask, MODEL_MAIN, EMAIL, DRY_RUN

# -----
# STEP 0. The budget and the rubric
# -----
MAX_ROUNDS = 2                                # the budget: the editor gets at most two looks

RUBRIC = (
    "You are an editor at Kaveri Freight. Check the draft below against this rubric:\n"
    " 1. Tone courteous?\n"
    " 2. Facts only, no promises of refunds or compensation?\n"
    " 3. Under 120 words?\n"
    "If all three pass, reply with exactly the single word PASS. "
    "Otherwise reply with ONE line of specific criticism and nothing else.\n\n"
    "Draft:\n"
)
# -----
# STEP 1. The loop: draft once, then (evaluate, maybe revise) up to MAX_ROUNDS times
# -----

def draft_and_polish(email_text):

    # 1a. WRITER, first draft. Loose instructions on purpose, so the editor has something to catch.
    draft = ask(MODEL_MAIN, "Reply to this customer exception email on behalf of Kaveri Freight:\n\n" +
        email_text)
    print("Draft 0:", len(draft.split()), "words")

    # 1b. The editing rounds. range(1, MAX_ROUNDS + 1) is the budget.
    for round_number in range(1, MAX_ROUNDS + 1):
        # EDITOR: PASS, or one line of criticism.
        verdict = ask(MODEL_MAIN, RUBRIC + draft)
        print("Round", round_number, "editor said:", verdict)

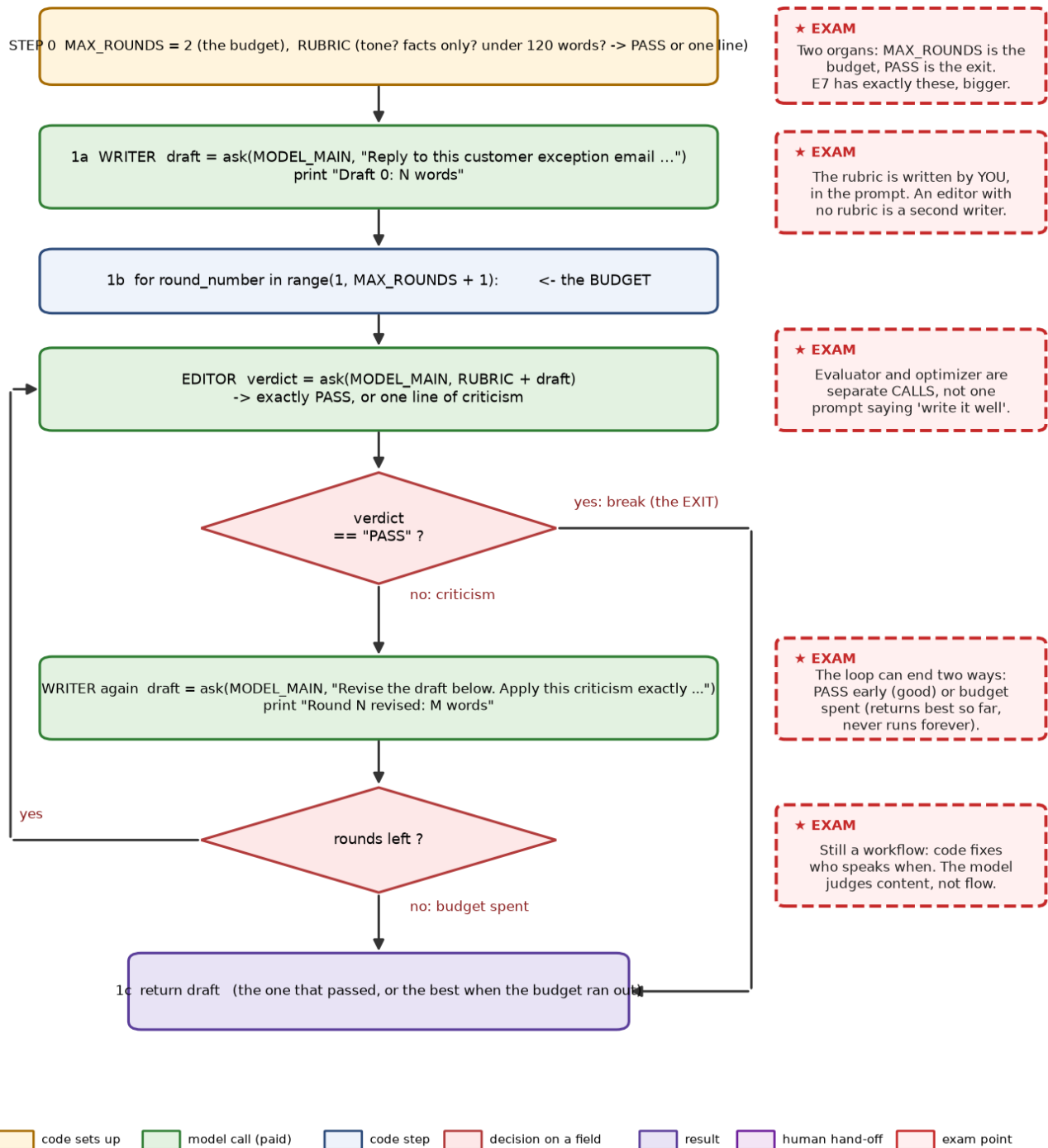
        # The exit. PASS means stop early and do not spend the remaining budget.
        if verdict == "PASS":
            break

        # WRITER again: revise using the criticism.
        draft = ask(MODEL_MAIN,
            "Revise the draft below. Apply this criticism exactly and change nothing else:\n"
            + verdict + "\n\nDraft:\n" + draft)
        print("Round", round_number, "revised:", len(draft.split()), "words")
    # 1c. Whatever draft we hold now ships: the one that passed, or the best after the budget ran out.
    return draft

if __name__ == "__main__":
    print("# E6 · evaluator-optimizer · " + ("DRY_RUN" if DRY_RUN else "LIVE"))
    final = draft_and_polish(EMAIL)
    print()
    print("=== Final draft ===")
    print(final)
```

E6 Evaluator-optimizer: the flowchart

E6 e6_evaluator_optimizer.py: writer, editor, a budget and an exit



Exam points on this chart

- ★ Two organs: MAX_ROUNDS is the budget, PASS is the exit. E7 has exactly these, grown larger.
- ★ The rubric is yours and lives in the prompt. Evaluator and optimizer are separate calls.
- ★ The loop ends two ways: PASS early, or budget spent with the best draft so far.

E7 The agent loop: the code

A bounded loop steered by stop_reason, every value on its own branch.

```
# E7 · The agent loop: every stop_reason lands somewhere explicit (objective 1.3c, the 50 percent row)
#
# A loop around messages.create that keeps going while the model asks for tools and stops when it does not.
# The field that steers it is stop_reason. BUDGET is written before the loop exists. Escalation is the floor.
#
# Run on battery: DRY_RUN=1 python e7_agent_loop.py
#                 D1_SCENARIO=default|truncate|refuse|exhaust makes the stand-in exercise each branch
# Run live:       python e7_agent_loop.py

import json
from common import make_client, trace, text_of, fake_status, fake_carriers, MODEL_MAIN, MAX_TOKENS,
    TOOLS_KAVERI, DRY_RUN, SCENARIO
# -----
# STEP 0. Things we decide BEFORE the loop exists
# -----

BUDGET = 6                                # the most turns we will ever pay for. Never "while True".
TOOLS = TOOLS_KAVERI                      # the detective's toolbag: both kaveri tools
BRIEF_E7 = "Why is KF-2481 stuck, and what are our options?"

# -----
# STEP 1. The tools, dispatched by name. Plain Python.
#         An unknown tool returns a structured error, not an exception: the model can read it and adapt.
# -----
def run_tool(tool_request):
    if tool_request.name == "get_shipment_status":
        return fake_status(tool_request.input["consignment_id"])
    if tool_request.name == "find_alternate_carriers":
        return fake_carriers(tool_request.input["lane"])
    return {"error": "unknown tool " + tool_request.name}

# -----
# STEP 2. The floor. When the loop cannot finish, a human gets the case WITH a reason.
# -----
def escalate(reason):
    return "ESCALATED to Meera's desk (" + reason + "): " + BRIEF_E7
# -----
# STEP 3. The loop
# -----
def investigate():
    client = make_client()
    conversation = [{"role": "user", "content": BRIEF_E7}] # the case file. The API is stateless, so this IS
    the memory.
    text_so_far = ""                                     # text that arrived before a max_tokens cut

    for turn in range(1, BUDGET + 1):                    # bounded, always
        reply = client.messages.create(
            model=MODEL_MAIN,
            max_tokens=MAX_TOKENS,
            tools=TOOLS,
            messages=conversation,
        )
        trace(turn, reply)                               # one line per turn, so you can watch the loop
        breathe

        # Collect any text in this reply.
        text_this_turn = ""
        for block in reply.content:
            if block.type == "text":
                text_this_turn = text_this_turn + block.text
        # ---- Every stop_reason value gets its own branch. No "else" that swallows. ----

        if reply.stop_reason == "tool_use":
            # The loop's fuel. Run every tool the model asked for, send all the results back.
            conversation.append({"role": "assistant", "content": reply.content})
            results = []
            for block in reply.content:
                if block.type == "tool_use":
                    results.append({
                        "type": "tool_result",
```

```

        "tool_use_id": block.id,
        "content": json.dumps(run_tool(block)),
    })
    conversation.append({"role": "user", "content": results})
    continue
if reply.stop_reason == "end_turn":
    # The natural exit. Return everything, including text saved from an earlier max_tokens cut.
    return text_so_far + text_this_turn

if reply.stop_reason == "max_tokens":
    # Truncated is not done. Keep the partial text, ask the model to continue.
    text_so_far = text_so_far + text_this_turn
    conversation.append({"role": "assistant", "content": reply.content})
    conversation.append({"role": "user", "content": "Continue."})
    continue

if reply.stop_reason == "pause_turn":
    # A server-side tool paused the turn. Send the reply back as it is to resume.
    conversation.append({"role": "assistant", "content": reply.content})
    continue
if reply.stop_reason == "refusal":
    # The model declined. A different path, never a blind retry.
    return escalate("model refused")

# Anything else: stop_sequence, context window exceeded, a value that does not exist yet.
return escalate("unhandled stop_reason " + str(reply.stop_reason))

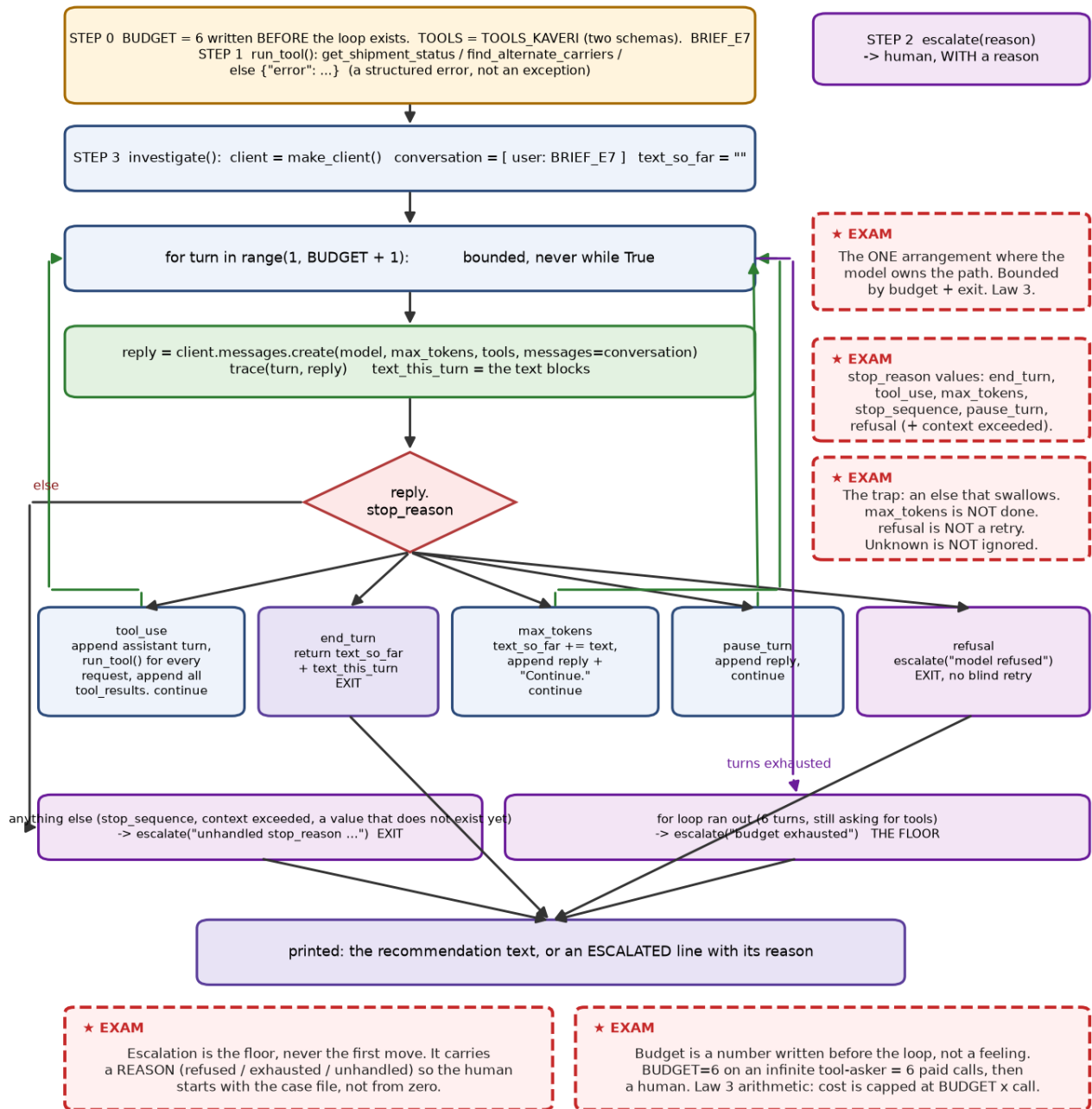
# The for loop ran out. The case is still open. A human takes it, with the reason.
return escalate("budget exhausted")

if __name__ == "__main__":
    print("# E7 · agent loop · " + ("DRY_RUN" if DRY_RUN else "LIVE") + " · scenario=" + SCENARIO + " · BUDGET="
          + str(BUDGET))
    print(investigate())

```

E7 The agent loop: the flowchart

E7 e7_agent_loop.py: every stop_reason lands somewhere explicit



code sets up model call (paid) code step decision on a field result human hand-off exam point

Exam points on this chart

- ★ The one arrangement where the model owns the path, bounded by budget and exit. Law 3.
- ★ Values: end_turn, tool_use, max_tokens, stop_sequence, pause_turn, refusal, and context exceeded.
- ★ The trap is an else that swallows. max_tokens is not done, refusal is not a retry, escalation carries a reason.

E8 Re-planning: the code

E5 plus a check after every finding: does the remaining plan still hold?

```
# E8 · Re-planning: the bridge is out, a finding can change the plan (objective 1.5, dynamic decomposition)
#
# Same three moves as E5 (make_plan -> run_worker -> synthesise), plus after EVERY finding the foreman is
# asked:
# "does the remaining plan still make sense?" If not, drop some tasks and add others.
# A second budget, MAX_NEW, caps how many tasks the plan may grow by. Law 3 applies to plans, not just turns.
#
# Run on battery: DRY_RUN=1 python e8_replanning.py          Run live: python e8_replanning.py

import json
from common import ask_json, MODEL_MAIN, BRIEF, DRY_RUN
from e5_orchestrator_workers import make_plan, run_worker, synthesise      # E5's three moves, reused
# -----
# STEP 0. The growth budget
# -----

MAX_NEW = 3                                # at most 3 tasks may ever be added to the plan

# -----
# STEP 1. The new move: after a finding, ask whether the remaining plan still holds.
# The foreman sees the new finding and the REMAINING tasks, not the whole history.
# -----
def check_plan(finding, remaining_tasks):
    question = (
        "A worker just reported this finding:\n" + finding + "\n\n"
        "These subtasks are still to be done:\n" + json.dumps(remaining_tasks) + "\n\n"
        "Does the remaining plan still make sense given the finding? Reply with JSON only:\n"
        ' {"ok": true}\n'
        "or\n"
        ' {"ok": false, "drop": [ids of tasks that no longer make sense], "add": ["short name of a new task",
        ...]}'
    )
    return ask_json(MODEL_MAIN, question)
# -----
# STEP 2. The loop with re-planning
# -----

def replan(brief):
    tasks = make_plan(brief)                # E5's opening move
    print("Initial plan:", json.dumps(tasks))

    findings = {}
    grew = 0                                # how many tasks have been added so far
    next_id = max(task["id"] for task in tasks) + 1    # fresh ids for added tasks

    while tasks:
        current = tasks.pop(0)               # keep going until the task list is empty
        result = run_worker(current, findings)    # take the first remaining task
        findings[current["id"]] = result        # E5's worker call
        print("Run", current["id"], "(" + current["task"] + ") ->", result)
        verdict = check_plan(result, tasks)      # the walk-and-look

        if verdict.get("ok", True):
            continue                          # plan still holds, move on

        if grew >= MAX_NEW:
            print(" Replan requested, but the growth budget is spent. Keeping the plan as it is.")
            continue

        # Drop the tasks the foreman says no longer make sense.
        drop_ids = set(verdict.get("drop", []))
        tasks = [task for task in tasks if task["id"] not in drop_ids]
        # Add the new ones as proper task records, but never past the cap.
        room_left = MAX_NEW - grew
        new_names = verdict.get("add", []):room_left]
        for name in new_names:
            tasks.append({"id": next_id, "task": name, "needs": []})
            next_id = next_id + 1
        grew = grew + len(new_names)
        print(" REPLAN: dropped", sorted(drop_ids), "added", new_names)

    if grew >= MAX_NEW:
        print("grew=" + str(grew) + ", budget MAX_NEW=" + str(MAX_NEW) + " EXHAUSTED")
```

```

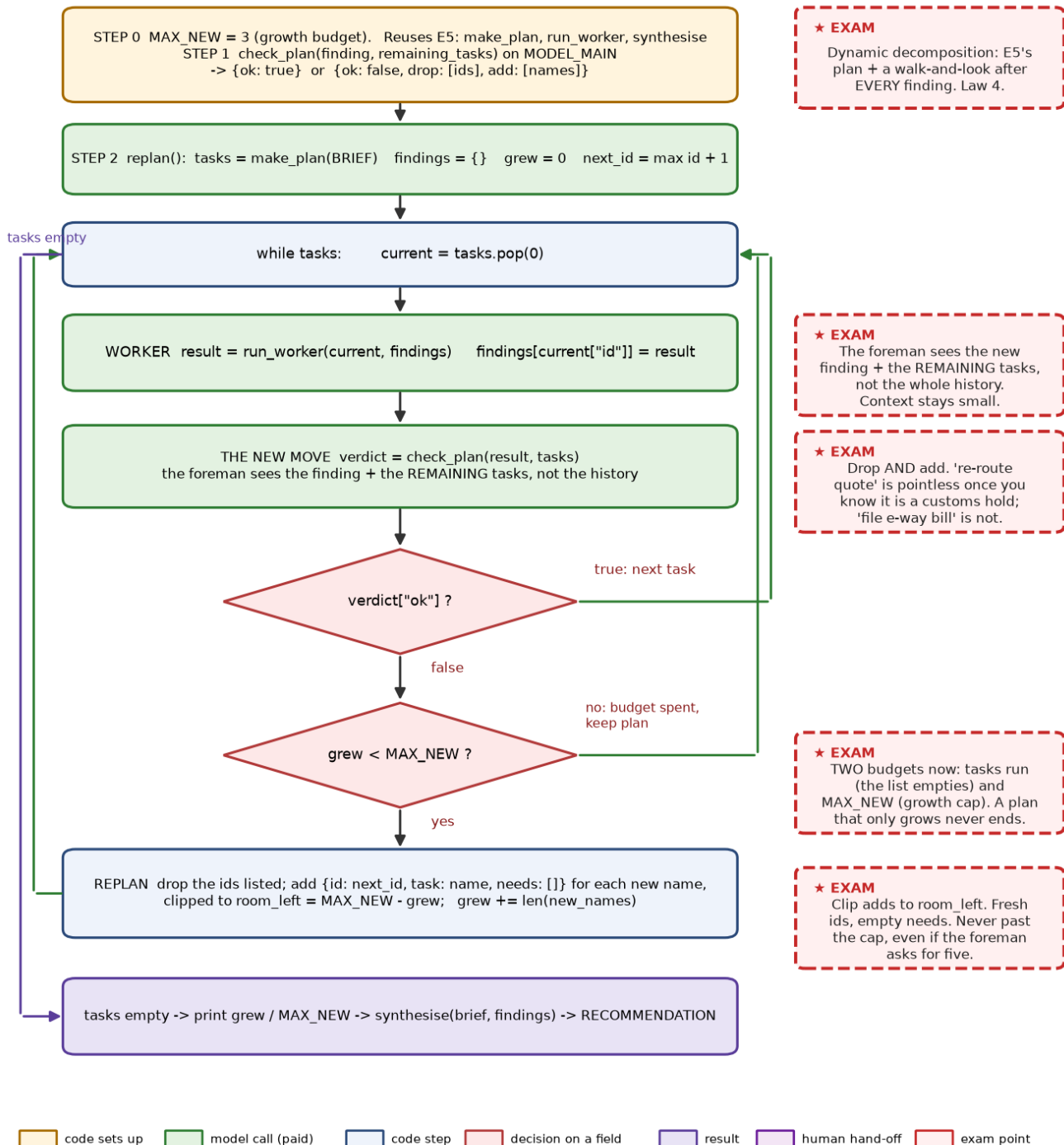
else:
    print("grew=" + str(grew) + ", budget MAX_NEW=" + str(MAX_NEW) + " not exhausted")
    return synthesise(brief, findings) # E5's closing move

if __name__ == "__main__":
    print("# E8 · re-planning · " + ("DRY_RUN" if DRY_RUN else "LIVE"))
    print(replan(BRIEF))

```

E8 Re-planning: the flowchart

E8 e8_replanning.py: a finding can change the plan, inside a growth budget



Exam points on this chart

- ★ Dynamic decomposition. Law 4. The foreman sees the new finding and the remaining tasks only.
- ★ Drop and add, with fresh ids and empty needs.
- ★ Two budgets: the task list empties, and MAX_NEW caps growth. A plan that only grows never ends.

Appendix standin.py: the dry-run stand-in

This is what answers when DRY_RUN=1. It reads the conversation and replies in character so the exhibits can run their real control flow without a key. It is not part of the patterns being taught; it is the battery.

```
# standin.py - a scripted stand-in for the Messages API, used when DRY_RUN=1.
#
# You do not need to read this file to learn the patterns. It exists so every exhibit can run its real
# control flow (the same stop_reason branches, the same replayed transcript) with zero paid calls.
# It reads the conversation it is given and answers in character:
# - with tools attached, it asks for the first tool that has no result yet, then gives a finale
# - without tools, it matches a few words in the prompt and returns a canned reply
# D1_SCENARIO=default|truncate|refuse|exhaust changes the finale so E7's branches can be watched.

import itertools
import json
import os
import re
from types import SimpleNamespace
SCENARIO = os.environ.get("D1_SCENARIO", "default")

# -----
# The canned world: fixed texts the stand-in hands back
# -----

DRAFT_FIRST = ("Dear Ms Deshmukh, thank you for reaching out to us regarding consignment KF-2481, and please
    accept "
        "our sincere apologies for the delay and for the lack of detail on the portal. I have checked the
        "
        "consignment personally and can confirm it is currently held at the Chennai inland container
        depot "
        "for a customs document check; specifically, the e-way bill attached to the shipment did not
        match "
        "the invoice value. Our team is re-filing the corrected bill today and we expect the depot to
        release "
        "the cartons within twenty-four hours of acceptance. Because of the inconvenience this has caused
        you "
        "and your buyer, we will refund your shipping charges in full and upgrade your next three
        shipments "
        "to express at no cost. I will send you the revised ETA personally as soon as the officer
        confirms "
        "release. Warm regards, KAVI for Kaveri Freight.")
CRITICISM = ("Over 120 words, and 'we will refund your shipping charges in full' promises compensation the desk
    "
        "has not approved.")
DRAFT_GOOD = ("Dear Ms Deshmukh, thank you for your message about KF-2481, and I am sorry for the delay and the
    "
        "silence on the portal. The consignment is held at the Chennai depot for a customs document check:
        "
        "the e-way bill did not match the invoice value. We are re-filing the corrected bill today, and
        the "
        "depot normally releases within a day of accepting it. I will send you the revised delivery time
        the "
        "moment the officer confirms. Your buyer may quote reference KF-2481 to us directly if that helps.
        "
        "Regards, KAVI for Kaveri Freight.")
PLAN_JSON = ('[{"id": 1, "task": "status check", "needs": []}, '
    '{"id": 2, "task": "carrier options", "needs": [1]}, '
    '{"id": 3, "task": "re-route quote", "needs": [2]}]')
FINDINGS = {
    "status check": "CUSTOMS_HOLD at Chennai depot; missing e-way bill; 3 days late",
    "carrier options": "BlueDart 2 days, Delhivery 3 days on Pune-Chennai; both need customs release first",
    "re-route quote": "Re-route quote Rs 2,400 via BlueDart; only useful after customs release",
    "file e-way bill": "Corrected e-way bill filed per SOP 214 section 4; expected release within 24 h",
    "notify padma": "Draft ready: revised ETA Saturday, goodwill note, no compensation promised",
}
RECOMMENDATION = ("RECOMMENDATION: file the e-way bill today; hold the re-route (costlier than the 24h release);
    "
        "send Padma the revised ETA with a goodwill note.")
FINAL_E7 = ("KF-2481 is held at customs for a missing e-way bill. Options: file the bill (fastest, release ~24
    h), "
        "or re-route via BlueDart/Delhivery once released. Recommend filing today and telling Padma the
        revised ETA.")
_votes = itertools.count() # E4 voting: APPROVE, REJECT, APPROVE -> majority APPROVE
_rubric = itertools.count() # E6: the first evaluation criticises, the second says PASS
def canned_text(system, prompt):
    """Match a few words in the system prompt and the user prompt, return the matching canned reply."""
```

```

p = (system or "").lower() + "\n" + prompt.lower()

if "one word only" in p:                                # E2 desk 1, E3 nurse
    return "customs"
if "approve or reject" in p:                            # E4 voting
    return "REJECT" if next(_votes) % 3 == 1 else "APPROVE"
if "eta for" in p:                                       # E4 scouts
    m = re.search(r"via (\w+)", prompt)
    carrier = m.group(1) if m else "carrier"
    return carrier + ": Thu 16:00, held at customs until release"
if "rubric:" in p:                                       # E6 editor
    return CRITICISM if next(_rubric) == 0 else "PASS"
if "revise the draft" in p:                             # E6 writer, second pass
    return DRAFT_GOOD
if "break this into" in p:                              # E5 foreman
    return PLAN_JSON
if "remaining plan" in p:                                # E8 check after a finding
    if "customs_hold" in p and "re-route" in p:
        return json.dumps({"ok": False, "drop": [3], "add": ["file e-way bill", "notify Padma of revised ETA"]})
    return json.dumps({"ok": True})
if "recommendation for meera" in p:                    # E5 and E8 synthesis
    return RECOMMENDATION
if "subtask:" in p:                                     # E5 and E8 workers
    m = re.search(r"subtask: (.+)", p)
    name = m.group(1).strip() if m else ""
    for key, value in FINDINGS.items():
        if name.startswith(key):
            return value
    return "no finding"
if "draft a courteous reply" in p:                     # E2 desk 2
    return DRAFT_GOOD
if "exception email" in p:                             # E6 first draft (deliberately too long, promises a refund)
    return DRAFT_FIRST
if "refund disputes" in p:                             # E3 wards
    return "Refund ward: claim acknowledged; verifying the damage note and insurance cover before any decision. No promise made."
if "customs holds" in p:
    return "Customs ward: KF-2481 is held for an e-way bill mismatch. Next steps: re-file the corrected bill (SOP 214 section 4), expect release within 24 h."
if "delay apologies" in p:
    return "Delay ward: We are sorry KF-2481 is late. New ETA: Saturday. We will confirm once the depot releases it."
return "dry-run reply"

# -----
# Reply objects shaped like the SDK's: .content (a list of blocks), .stop_reason, .usage
# -----

def _block(**fields):
    return SimpleNamespace(**fields)

def _reply(blocks, stop_reason, **extra):
    return SimpleNamespace(content=blocks, stop_reason=stop_reason,
                           usage=SimpleNamespace(input_tokens=0, output_tokens=0), **extra)

def _field(block, name):
    """Read a content-block field whether the block is an SDK object or a plain dict. The API takes both."""
    if isinstance(block, dict):
        return block.get(name)
    return getattr(block, name, None)

class _FakeMessages:
    def __init__(self):
        self.calls = 0

    def create(self, *, model, max_tokens, messages, system=None, tools=None, **_):
        self.calls = self.calls + 1
        if tools:
            return self._with_tools(messages, system or "", tools)
        # No tools: answer the last user message from the canned world.
        last = messages[-1]["content"]
        if isinstance(last, str):
            prompt = last
        else:
            prompt = " ".join(b.get("text", "") for b in last if isinstance(b, dict))
        return _reply([_block(type="text", text=canned_text(system, prompt))], "end_turn")
    def _with_tools(self, messages, system, tools):

```

```

# Read the stateless transcript: which tool requests already have a result?
id_to_name = {}
answered = set()
for message in messages:
    content = message["content"]
    if message["role"] == "assistant" and not isinstance(content, str):
        for block in content:
            if _field(block, "type") == "tool_use":
                id_to_name[_field(block, "id")] = _field(block, "name")
    if message["role"] == "user" and isinstance(content, list):
        for block in content:
            if isinstance(block, dict) and block.get("type") == "tool_result":
                answered.add(id_to_name.get(block["tool_use_id"]))
transcript = json.dumps(messages, default=str)
found = re.search(r"KF-\d{4}", transcript)
consignment_id = found.group() if found else "KF-2481"

if SCENARIO == "exhaust":
    # a detective who never stops asking for the phone
    return _reply([_block(type="tool_use", id="toolu_dry_" + str(self.calls),
                        name="get_shipment_status", input={"consignment_id": consignment_id}),
                  "tool_use"])

for tool in tools:
    # the first tool with no result yet gets called
    if tool["name"] not in answered:
        if tool["name"] == "get_shipment_status":
            tool_input = {"consignment_id": consignment_id}
        else:
            tool_input = {"lane": "Pune-Chennai"}
        return _reply([_block(type="tool_use", id="toolu_dry_" + str(self.calls),
                        name=tool["name"], input=tool_input)], "tool_use")
# Every tool answered: the finale, steered by the scenario.
if SCENARIO == "refuse":
    return _reply([], "refusal",
                  stop_details=SimpleNamespace(type="refusal", category="general_harms",
                                                explanation="scripted dry-run refusal"))

if "triage" in system.lower():
    final = '{"category": "customs", "urgency": "high"}'
else:
    final = FINAL_E7

continued = False
for message in messages:
    if message["role"] == "user" and message["content"] == "Continue.":
        continued = True
if SCENARIO == "truncate" and not continued:
    cut = len(final) // 2
    return _reply([_block(type="text", text=final[:cut])], "max_tokens")
if SCENARIO == "truncate" and continued:
    return _reply([_block(type="text", text=final[len(final) // 2:]), "end_turn")
return _reply([_block(type="text", text=final)], "end_turn")

class FakeClient:
    def __init__(self):
        self.messages = _FakeMessages()

```